RayatShikshanSanstha's

R.B.Narayanrao Borawake College, Shrirampur

(Autonomous)

National Education Policy

(Affiliated to Savitribai Phule Pune University)

Two Years Degree Program in Computer Science(Faculty of Science and Technology)

Syllabus under Autonomy and National Education Policy

M.Sc.(ComputerScience)Part-II

Practical Assignment LabBook

Choice Based Credit System[CBCS]Syllabus for National Education Policy to be implemented from Academic Year 2024-2025

Artificial Intelligence(CS-MJ-515P)

Assignment Completion Sheet

Name of the Student:

Roll No:

Sr.No.	AssignmentTitle	Marks Obtained	Signature of Instructor
1	Program to print multiplication table for given no.		
2	Accept number and check prime or not		
3	Accept number and print factorial or not		
4	List operation: concatenantion,iteration,nestedlist,extend,append		
5	Set operation:intersection,difference,symmetric difference,		
6	Write a program to implement Breadth First Search Traversal		
7	Write a program to implement Depth First Search Traversal.		
8	Write a program to implement Water Jug Problem		
9	Develop a program to solve the eight queens problem. (Uninformed Search)		
10	Develop a program to solve the N queens problem. (Uninformed Search)		
11	Write a computer program to play tic-tac-toe game. (Game Theory)		
12	Write a program to implement Simple Chatbot.		

Total Marks:

Converted into10Marks:
Date:

Signature of InCharge **Head of Department**

InternalExaminer **External Examiner**

AssignmentNo1

Q.1) Program to print multiplication table for given no.

```
n=3
for i in range(1,11):
print(n*i)
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.2

Q.2)Accept number and check prime or not.

```
flag=0
n=int(input("enter num"))
for i in range(2,n+1):
        if(n%2==0):
         flag=1
if(flag==1):
        print("not prime")
else:
        print("prime")
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.3

Q.3)Accept number and print factorial or not.

```
n=int(input("enter num"))
f=1
for i in range(1,n+1):
        f=f*i
print(f)
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.4

Q.4) List operation: concatenantion,iteration,nestedlist,extend,append.

```
l1=[10,20,30,40]
print(l1)

#concatenantion
list1=[11,22,33]
list2=[10,20,30]
print(list1)

#nested list
list=[[10,20,30],['aa','gg']]
list

#concatenantion
list1=[11,22,33]
list2=[10,20,30]
print(list1)

#iteration
l4=[10,40,90]
for i in l4:
    print(i)

#add
l4.append(50)
print(l4)

#extend
s1=['pen','pencil']
s2=['apple','notebook']
s1.extend(s2)
print(s1)
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.5

Q.5) Set operation:intersection,difference,symmetric difference.

```
s1 = {0, 2, 4, 6, 8};
s2= {1, 2, 3, 4, 5};

# set union
print("Union of s1 and s2 is",s1 | s2)

# set intersection
print("Intersection of s1 and s2 is",s1& s2)

# set difference
print("Difference of s1 and s2 is",s1 - s2)

# set symmetric difference
print("Symmetric difference of s1 and s2 is",s1 ^ s2)
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.6

Q.6)Write a program to implement Breadth First Search Traversal.

```
graph = {
  '10' : ['20','30'],
  '20' : ['40', '50'],
  '30' : ['60','70'],
  '40' : [],
  '50' : [],
  '60' : [],
  '70':[]
}

visited = [] # List for visited nodes.
queue = []     #Initialize a queue

def bfs(visited, graph, node): #function for BFS
  visited.append(node)queue.append(node)
while queue:
        m = queue.pop(0)
        print (m, end = " ")
for neighbour in graph[m]:
        if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)
# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, '10') # function calling
```

AssignmentEvaluation

0:NotDone		1:Incomplet		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo7

Q.7)Write a program to implement Depth First Search Traversal.

```
graph = {
    '5' : ['3','7'],
    '3' : ['2', '4'],
    '7' : ['8'],
    '2' : [],
    '4' : ['8'],
    '8' : []
}

visited = set() # Set to keep track of visited nodes of graph.

def dfs(visited, graph, node):   #function for dfs
        if node not in visited:print (node)
                visited.add(node)
                for neighbour in graph[node]:
                        dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.8

Q.8)Write a program to implement Water Jug Problem.

```
from collections import deque
def water_jug_problem_trace_path(jug1_capacity, jug2_capacity, target):
# Queue to keep track of states and the path to reach them

queue = deque([((0, 0), [(0, 0)])])
# Set to keep track of visited states
visited = set()
visited.add((0, 0))

while queue:
    (jug1, jug2), path = queue.popleft()

# Check if we've reached the target
if jug1 == target or jug2 == target:
    return path
# Possible states from the current state possible_states = [
(jug1_capacity, jug2), # Fill jug1
(jug1, jug2_capacity), # Fill jug2
(0, jug2), # Empty jug1
(jug1, 0), # Empty jug2
(max(0, jug1 - (jug2_capacity - jug2)), min(jug2_capacity, jug1 + jug2)),  # Pour jug1 to jug2
(min(jug1_capacity, jug1 + jug2), max(0, jug2 - (jug1_capacity - jug1))) # Pour jug2 to jug1
]
for new_jug1, new_jug2 in possible_states:
    new_state = (new_jug1, new_jug2)
    if new_state not in visited:
        visited.add(new_state)
        queue.append((new_state, path + [new_state]))
return None # If no solution is found

# Example usage: result = water_jug_problem_trace_path(4, 3, 2)
```

```
if result:
    print("Path to target:", result)
else:
    print("No solution found.")
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.9

Q.9)Develop a program to solve the eight queens problem. (Uninformed Search)

```
N = 8 # (size of the chessboard)

def solveNQueens(board, col):
if col == N:
        print(board)
        return True
for i in range(N):
        if isSafe(board, i, col):
                board[i][col] = 1
                if solveNQueens(board, col + 1):
                        return True
                board[i][col] = 0
return False

def isSafe(board, row, col):
        for x in range(col):
                if board[row][x] == 1:
                        return False
        for x, y in zip(range(row, -1, -1), range(col, -1, -1)):
                if board[x][y] == 1:
                        return False
        for x, y in zip(range(row, N, 1), range(col, -1, -1)):
                if board[x][y] == 1:
                        return False
        return True

board = [[0 for x in range(N)] for y in range(N)]
if not solveNQueens(board, 0):
        print("No solution found")
```

AssignmentEvaluation

0:NotDone		1:Incomplete		2:Late Complete	
3:NeedImprovement		4:Completed		5:WellDone	

Date: **PracticalIn-charge**

AssignmentNo.10

Q.10)Develop a program to solve the N queens problem. (Uninformed Search)

```
global N N = 4

def printSolution(board):
        for i in range(N):
                for j in range(N):
                        print (board[i][j],end=' ')
                        print()


# A utility function to check if a queen can
# be placed on board[row][col]. Note that this # function is called when "col" queens are
# already placed in columns from 0 to col -1. # So we need to check only left side for
# attacking queens
def isSafe(board, row, col):

# Check this row on left side
for i in range(col):
        if board[row][i] == 1:
                return False

# Check upper diagonal on left side
for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
                return False

# Check lower diagonal on left side
for i, j in zip(range(row, N, 1), range(col, -1, -1)):
        if board[i][j] == 1:
                return False
return True

def solveNQUtil(board, col):
# base case: If all queens are placed # then return true
if col >= N:
        return True
```

```
# Consider this column and try placing # this queen in all rows one by one
for i in range(N):

        if isSafe(board, i, col):
        # Place this queen in board[i][col]
        board[i][col] = 1

        # recur to place rest of the queens
        if solveNQUtil(board, col + 1) == True:
                return True

# If placing queen in board[i][col # doesn't lead to a solution, then

# queen from board[i][col]
board[i][col] = 0

# if the queen can not be placed in any row in # this column col then return false
return False

# This function solves the N Queen problem using # Backtracking. It mainly uses solveNQUtil()
to # solve the problem. It returns false if queens # cannot be placed, otherwise return true and
# placement of queens in the form of 1s. # note that there may be more than one
# solutions, this function prints one      of the # feasible solutions.

def solveNQ():
board = [ [0, 0,0,0],
[0,0,0,0],
[0,0,0,0],
[0,0,0,0]
]

if solveNQUtil(board, 0) == False:
        print ("Solution does not exist")
        return False

printSolution(board)
        return True
```

# driver program to test above function
solveNQ()

Assignment Evaluation

0:Not Done		1:Incomplete		2:Late Complete	
3:Need Improvement		4:Completed		5:Well Done	

Date: **PracticalIn-charge**

AssignmentNo.11

Q.11)Write a computer program to play tic-tac-toe game. (Game Theory)

```
board = ["-", "-", "-",
         "-", "-", "-",
         "-", "-", "-"]
# Define a function to print the game board
def print_board():
 print(board[0] + " | " + board[1] + " | " + board[2])print(board[3] +
 " | " + board[4] + " | " + board[5])print(board[6] + " | " + board[7]
 + " | " + board[8])
# Define a function to handle a player's turn
def take_turn(player):
 print(player + "'s turn.")
 position = input("Choose a position from 1-9: ")
 while position not in ["1", "2", "3", "4", "5", "6", "7", "8", "9"]:position = input("Invalid
input. Choose a position from 1-9: ")
 position = int(position) - 1
 while board[position] != "-":
  position = int(input("Position already taken. Choose a different positboard[position]
 = player
 print_board()
# Define a function to check if the game is over
def check_game_over():
 # Check for a win
 if (board[0] == board[1] == board[2] != "-") or \(board[3] ==
  board[4] == board[5] != "-") or \
  (board[6] == board[7] == board[8] != "-") or \
  (board[0] == board[3] == board[6] != "-") or \
  (board[1] == board[4] == board[7] != "-") or \
  (board[2] == board[5] == board[8] != "-") or \
  (board[0] == board[4] == board[8] != "-") or \
  (board[2] == board[4] == board[6] != "-"):
   return "win"
 # Check for a tie
 elif "-" not in board:
  return  "tie" # Game is
 not overelse:
```

```
            return "play"
    # Define the main game loop
        Def
    play_game(): print_board()
          current_player = "X"game_over
          = False
          while not game_over:

               game_result == "win":
                    print(current_player + " wins!")game_over =
                    True
               elif game_result == "tie":print("It's a
                    tie!")
                    game_over = True
             else:
                  # Switc1h to the other player
                  current_player = "O" if current_player == "X" else "X"

# Start the game
play_game()
```

AssignmentEvaluation

0:Not Done		1:Incomplete		2:Late Complete	
3:Need Improvement		4:Completed		5:Well Done	

Date: **PracticalIn-charge**

AssignmentNo.12

Q.12) Write a program to implement Simple Chatbot.

```
responses={"hello":"hello",
           "hii":"how are u",
           "bye":"byeeee"}
def get_responses(msg):
    msg=msg.lower()

    if msg in responses:
      print(msg)
      return responses[msg]

    else:
        return "i didn't understand"
while True:
    user_input=input("you: ")
    responses=get_responses(user_input)print("chatbot:
    ",responses)
```

AssignmentEvaluation

0:Not Done		1:Incomplete		2:Late Complete	
3:Need Improvement		4:Completed		5:Well Done	

Date: **PracticalIn-charge**